NMR Tuning and Operation

Quadrature Phase Tunes, Auto Tune and the New Run Controls in PyNMR (UNH fork)

UNH Polarized Target Group

November 4, 2026

Note on provenance. This document was drafted with AI assistance (Claude), under the direction of and with technical input from E. Long. It has not undergone the review process required for publication and may contain errors, gaps, or outdated statements. It is included here because it is useful for understanding and running the accompanying code, not as a citable or publication-ready reference.

Abstract

This note is the operator's guide to the tuning and quadrature features added to PyNMR in the UNH fork (`unhpoltarg/jlab_pynmr`) during October–November 2026. PyNMR can now record both quadratures of the deuteron NMR signal, absorption (Real, 0°) and dispersion (Imag, 90°), by stepping the phase detector's reference between events, optionally with the opposite-sign settings 180° and 270° that cancel detector offsets. The operator sets each phase tune roughly by eye; **Auto Tune** then moves them to the true angles using only the background Q-curve, in about 50 s for a Real/Imag pair. The note covers starting PyNMR safely, tuning, taking quadrature data, what lands in which file, and what to do when something goes wrong. An appendix explains why the Q-curve alone fixes the absorptive phase: at every frequency the NMR absorption points along the normal to the Q-curve's trace in the complex plane, which is the origin of the empirical form $\theta(f) = \theta_0 + 2\arctan\big((f - f_0)/\gamma\big)$. Everything here has been checked against the built-in simulator. **Auto Tune, the split plots, the per-angle baselines, the dispersive background and the $64 \times N$ sweep controls have not yet been run on the Hall B box.**

Quick Start Guide

For operators who have done this before. Each red number (n) on the screenshots (Figs. 1–3) is the step with that number. Everything after this guide is background and detail.

1. **Start PyNMR** on the DAQ computer, then start the online fitter if you use it:

   ```
   conda deactivate          # repeat until no environment is active
   conda activate pynmr312
   python pynmr_main.py -p Deuteron
   ```

2. (2) **Run tab:** set **Center Freq** (the Larmor frequency) and **Half-width**, then **Apply Window**. (In the `TestSimQuad` simulator the **Event Label** also drives the target: None/Baseline, TE, Polarize, Junk.)

3. (3) **Sweeps per Event** $= 64 \times N$: **Unlock**, set N (default 10), **Set**.
4. (4) **Tune tab: Run** the tune loop and tune the tank with **Diode Tune Control**.
5. (5) **Rough phase tunes:** select **Real (0°)** and set the **Phase Tune** by eye so the Q-curve looks absorptive; select **Imag (90°)** and set it about a quarter turn away (cycle4: also 180°, 270°). Within about 30° is enough. Last cooldown's tunes are loaded as defaults.
6. (6) **Auto Tune** (about 50 s; the tune loop pauses and restarts by itself).
7. (7) **Check** the result line (how far each tune was off, ± error) and the trace plot. Wrong? **Undo Auto Tune**.
8. (8) **Auto Calibrate Phase** (about 90 s), only at the start of a cooldown, after an RF hardware change, or after the centre frequency moved by more than 0.05 MHz.

   Sign: at the true 0° (diode smile, phase frown) positive P is a downward line. Take a TE there before trusting P: a CC from the old 180° setting has the wrong sign (§3).
9. (9) **Stop the tune loop.** On the Run tab choose the sequence (pair, abba or cycle4) and tick **Quadrature**. If it won't tick, the reason is shown in red under it.
10. (10) **Baseline run:** move the field so the line is out of the window, set Event Label to Baseline, **Run** a few complete cycles, **Stop**.
11. (11) **Baseline tab: Select Current Eventfile**; select the whole baseline run (click its first row, shift-click its last); read the summary line; **Set Baseline**.
12. (12) **Run tab:** the **Baselines** line must say ok for every angle.
13. **Take data:** field back on the line, Event Label as needed, **Run**.
14. (14) **Wing bins** (on Baseline Subtracted Real): adjust until the dashed red part of the yellow fit stays clear of the line.
15. **Stop** with **Stop**. If the exit dialog appears, choose **Abort! I didn't mean to make this appear!**

Also: a TE (§7); the Analysis tab (§8); anything in red (§9).

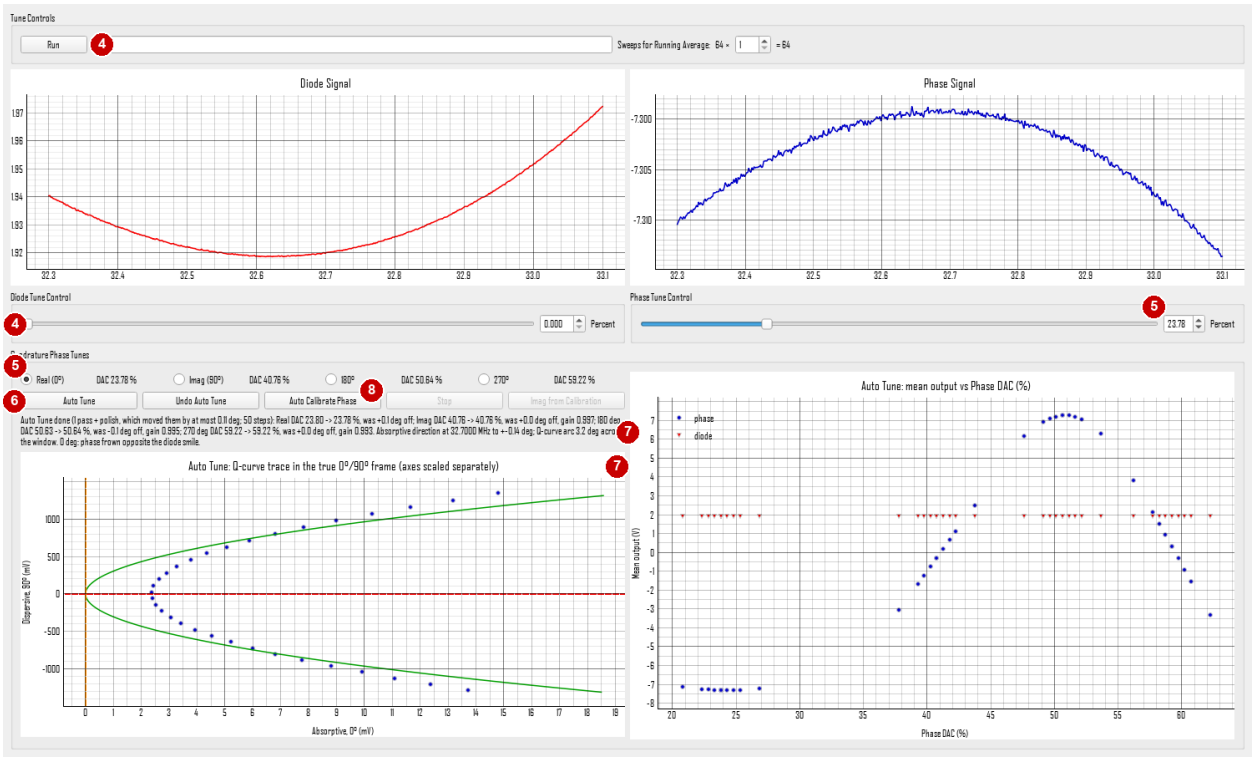


Figure 1: Quick Start, Tune tab. 4 **Run** the tune loop; **Diode Tune Control**. 5 the tune selector (**Real (0°)**, **Imag (90°)**, 180°, 270°, each with its phase DAC setting) and the **Phase Tune** box. 6 **Auto Tune** (**Undo Auto Tune** beside it). 7 the result line and, under it, the trace plot (red dashes: the 0° direction; orange: 90°). 8 **Auto Calibrate Phase**. Right: mean output against phase DAC for the scan.



Figure 2: Quick Start, Run tab (cycle4 run, simulator). 2 window: **Center Freq**, **Half-width**, **Apply Window**. 3 Sweeps per Event $64 \times N$. 9 **Quadrature** and the sequence. 10 **Run**. 12 the **Baselines** line. 14 **Wing bins**, with the fitted points dashed red on the yellow fit.

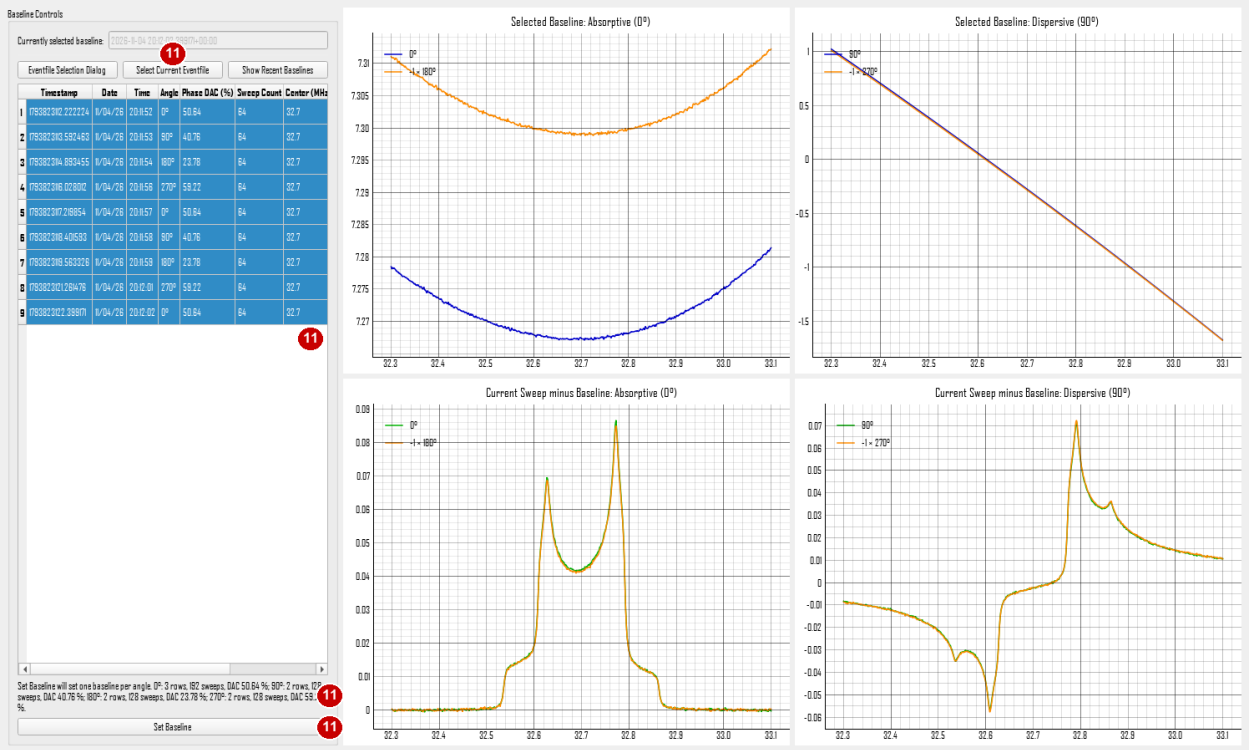


Figure 3: Quick Start, Baseline tab. 11 **Select Current Eventfile**; select all rows of the baseline run (the Angle and Phase DAC columns show each row's setting); check the summary line above **Set Baseline**; click it. Right: each angle's baseline (top) and the current sweep minus it (bottom), absorptive left, dispersive right.

Contents

Quick Start Guide 1

1 What is new 5

2 Before you start 5

3 Concepts 6

4 Setting up the NMR window and sweep counts 7
- 4.1 The NMR window . . . 7
- 4.2 Sweeps as $64 \times N$ . . . 7
- 4.3 The wing fit: points per side . . . 8

5 Tuning 8
- 5.1 Tuning procedure . . . 8
- 5.2 Auto Tune . . . 9
- 5.3 Auto Calibrate Phase . . . 11

6 Taking quadrature data 11
- 6.1 Starting a quadrature run . . . 11
- 6.2 Baselines . . . 12
- 6.3 Background subtraction for 90° and 270° records . . . 13
- 6.4 The plots . . . 14
- 6.5 Where the records go . . . 14

7 Taking and using a TE 15

8 The Analysis tab 16

9 Troubleshooting 18

10 Status and limits 18

A Why the Q-curve fixes the absorptive phase 19

B The Auto Tune algorithm 20

1 What is new

All UNH code lives in `unh/`; every change to a JLab file is marked with a `UNH-HOOK` label and can be removed with `python -m unh.strip_unh`. The quadrature code is in `unh/quadrature/`, its settings in `unh/quadrature/quadrature.yaml`, and its design notes in `notes/quadrature_phase_design.md`.

Feature	What it does for the operator	Where
Python 3.12 env	PyNMR on the DAQ computer runs in `pynmr312`; the old 3.11 env crashed mid-run	§2
Box connections	Each event closes its connection to the FPGA box cleanly; the box is told to stop after errors	§2
Data streamer	Each finished 0° event goes to the online fitter; on by default	§2
NMR window	Centre frequency and half-width set from the Run tab, between events	§4.1
$64 \times N$ sweeps	Sweeps per event and the Tune running average are whole 64-sweep chunks	§4.2
Wing points	The wing fit's size as points per side, with the fitted points marked	§4.3
Phase tunes	Separate Real, Imag, 180° and 270° tunes on the Tune tab	§5
Auto Tune	Refines the hand tunes to the true angles from the Q-curve	§5.2
Auto Calibrate Phase	Measures the phase shifter over its whole range	§5.3
Quadrature runs	Events alternate between tunes: pair, abba or cycle4	§6
Quadrature baselines	One selection of a baseline run gives each angle its own baseline	§6.2
Dispersive background	90°/270° records: background fitted jointly with KK of the absorption	§6.3
Quadrature plots	A 2×3 grid: each processing stage split into 0° and 90°	§6.4

Table 1: The new features and where they are described.

2 Before you start

Use the Python 3.12 environment. On the DAQ computer (Linux/WSL):

```
conda deactivate          # repeat until no environment is active
conda activate pynmr312
python --version          # must say 3.12 or newer
python pynmr_main.py -p Deuteron
```

On the old Python 3.11 environment PyNMR died silently in the middle of runs, most often with the streamer on. The cause was `Fatal Python error: none_dealloc`, raised while pyqtgraph redrew the plots; Python 3.12 cannot raise it. Stacked conda environments can leave the old Python first on the path, hence the `conda deactivate`. The fitter should also run on Python 3.12.

Never kill PyNMR in the middle of a sweep. The deuteron box (192.168.0.72) stops answering, even on UDP, if PyNMR dies while the box is sweeping, and only a power cycle brings it back. Stop runs with the **Run**/**Stop** button and close PyNMR normally. If you close PyNMR while the Run loop is on, it asks first: choose **Abort! I didn't mean to make this appear!** to keep running; **Ignore. I am an expert and I understand the consequences.** closes it mid-sweep. There is no such warning for the Tune loop, **Auto Tune** or **Auto Calibrate Phase**: stop those before closing. PyNMR now closes every connection cleanly and sends the stop-sweep command after errors and at the end of tuning; on the DAQ computer this cut the connection time from 2.0 s to under 0.2 s per event.

The online fitter. The data streamer is on by default (`unh/data_streamer.yaml`, `enable: True`). It sends each finished deuteron event's fit-subtracted line to LabView-NMR-Fitter on `127.0.0.1:8777`. If the fitter isn't running, PyNMR logs one warning per outage and carries on; it never waits for the fitter. In quadrature runs only the 0° records are sent.

Turning quadrature on for a profile. The quadrature controls appear only for profiles enabled in `unh/quadrature/quadrature.yaml`:

```
profiles:
    Deuteron:
        enable: True
```

The simulator profile `TestSimQuad` has them on already, and runs with no hardware: `python pynmr_main.py -p TestSimQuad`. In it the Run tab's **Event Label** decides what the simulated target does: *None* or *Baseline*, the line moved out of the window (P relaxes towards TE); *TE*, thermal equilibrium at 5 T and 1 K (about 0.1 %); *Polarize*, a spin-up from wherever P is; *Junk*, a very noisy, bouncing line that is useless for anything. The simulator has no temperature reading, so a TE can't be completed on the TE tab there.

3 Concepts

Phase-sensitive detection. The Q-meter's phase channel multiplies the tank voltage V by a reference of phase ϕ and keeps $P_\phi = g\,\mathrm{Re}\big(Ve^{-i\phi}\big) + o$, where g is the detector gain and o a DC offset. With the reference on the absorptive direction the output is the absorption χ'' (**Real,** 0°); a quarter turn later it is the dispersion χ' (**Imag,** 90°). At 180° and 270° the signal flips sign but the offset o does not, so the pair differences

$$I = \tfrac{1}{2}\big(P_0 - P_{180}\big), \qquad Q = \tfrac{1}{2}\big(P_{90} - P_{270}\big) \tag{1}$$

are free of detector offsets.

Which way is 0°: the UNH sign convention (E. Long, 2026-11-04). For the deuteron the $m = +1$ level is lowest, 0 in the middle and -1 highest. With positive P the spin system takes energy from the RF, so at the true 0° setting a positive polarization gives a *dip*, a downward-pointing line, and a negative polarization an upward one. On this Q-meter that 0° setting is the one where the phase Q-curve is a **frown** while the diode is a **smile**. The 2026-10-27 data agree: at phase DAC 0.535–0.547 (frown) the line pointed down, at 0.389–0.40 (smile) up. **Past UNH cooldowns ran at the** 180° **setting** (smile), with upward lines for positive P; their P values are still right wherever the CC came from a TE taken at the same setting, because the CC carries the sign. Auto Tune now enforces the frown (§5.2).

- **Take a new TE after moving to the true** 0°**.** A CC from a TE at the old 180° setting has the wrong sign there, so P reads with the wrong sign until a TE is taken at the new setting.
- The online fitter's McClellan model (and PyNMR's Optimized McClellan Fits) wants positive P as an *upward* line, so the streamer flips the line before sending (`line_sign: -1` in `unh/data_streamer.yaml`). Set it to +1 only for data taken at the old setting. The TestSim profile keeps +1.
- The TestSimQuad simulator has the UNH box's diode polarity (a smile) and starts on its frown side, so it behaves like the hardware.

The phase DAC. The reference phase is set by a Mini-Circuits voltage-controlled phase shifter (SCPHS-51+ on the deuteron box) driven by the box's phase DAC. The Tune tab shows the DAC setting as a percentage of full scale, for example "DAC 51.40 %". **This is a control setting, not a fraction of the signal.** The shifter covers about 456° at 32.7 MHz, but its curve is S-shaped (a few degrees per % near 0, about 10° per % in the middle) and it stops responding above about 80 % DAC. The curve also depends strongly on frequency: a calibration at 32.7 MHz is wrong at 22 MHz.

What the Q-curve can and can't tell. As the frequency is swept, the Q-curve traces a curve in the complex plane (a circle, or nearly a straight line for a heavily damped circuit). Appendix A shows that the NMR absorption at each frequency points along the normal to that trace. So the Q-curve alone fixes the absorptive direction, and **Auto Tune** uses that. Two things it can't fix are left to the hand tunes: whether "0" is θ or $\theta+180°$ (both are absorptive), and whether "90" is $+90°$ or $-90°$ (the sign of the rotation isn't observable from projections). Auto Tune keeps whichever is nearer the operator's hand setting.

The limit on 90°. Projections determine the reference direction only up to a 2×2 linear map, so a 90° angle needs one assumption: that the detector gain changes smoothly with the DAC setting. 0° and 180° (antiparallel) don't need it. PyNMR's numbers are good for choosing and monitoring the tunes; a 0.1° phase should come from the offline complex lineshape fit, which uses the Kramers–Kronig relation between χ' and χ''.

4 Setting up the NMR window and sweep counts

4.1 The NMR window

In NMR Settings on the Run tab, **Center Freq (MHz)** and **Half-width ± (kHz)** set the sweep window; **Apply Window** applies them. When PyNMR is idle the change is immediate; during a run it waits for the next event. The half-width can go up to 5000 kHz in the deuteron profiles: the R&S SMA100A's full FM deviation, $r_m \times 10$ MHz with $r_m = 1/2$ for carriers up to 46.875 MHz (datasheet v06.01, p. 14). Its deviation accuracy is specified up to half that, 2500 kHz. Other profiles keep 2000 kHz (`window_limits: hw_max_khz` in `pynmr_config.yaml`). The window is saved in the session and restored at the next start. Keep the centre on the deuteron Larmor frequency: **Auto Tune** finds the absorptive phase at the centre frequency.

4.2 Sweeps as $64 \times N$

The box delivers sweeps in chunks of 64, and when asked for 32 it returned 64. Sweep counts are therefore whole chunks:

- **Run tab, Sweeps per Event:** $64 \times N$, with N set by the operator (default 10, i.e. 640 sweeps). Press **Unlock**, change N, press **Set**, as before.
- **Tune tab, Sweeps for Running Average:** $64 \times N$, default $N = 1$. Larger N gives a smoother plot but makes it lag the knobs by about 1.8 s per chunk.
- The tune chunk itself (`tune_per_chunk` in `pynmr_config.yaml`) is 64 sweeps, so the Tune loop, **Auto Tune** and **Auto Calibrate Phase** all use 64-sweep chunks.

4.3 The wing fit: points per side

The yellow curve on Baseline Subtracted is JLab's *Polynomial Fit to Wings* (3rd order by default), which is subtracted to give Fit Subtracted. It is fitted to the outer parts of the sweep only. A box in the title row of the Baseline Subtracted plot, **Wing bins**, sets how many points (bins) it uses on *each* side; a dashed red curve on top of the yellow one marks exactly those points.

- The default is what the Analysis tab's fit bounds give (`analysis: wings` in `pynmr_config.yaml`): 148 points per side of 512 for the shared default 0.01/0.30/0.70/0.99, and 122 for the TestSim profiles. JLab's skip of the outermost 1 % on each side is kept.

- Changing N writes the Analysis tab's four bounds (they stay the one setting), redraws the last event's fit at once so the effect can be judged by eye, and applies to every event after it. Records already taken keep the fit they were analysed with.
- Choose N so the dashed red part stays clear of the line: too many points and the fit takes signal for background; too few and the polynomial is poorly pinned down.
- In quadrature the box sits on the Baseline Subtracted Real plot. The 90° plot is dashed over the whole sweep, because dispersive records use the joint KK fit (§6.3), not the wings.
- N goes back to the default when PyNMR restarts.

5 Tuning

Figure 4 shows the Tune tab. Below the diode and phase plots, the **Quadrature Phase Tunes** strip holds a selector for the tune being edited (Real, Imag, and 180°/270° when the sequence uses them) with each tune's DAC setting, the buttons, a result line, and a small plot.

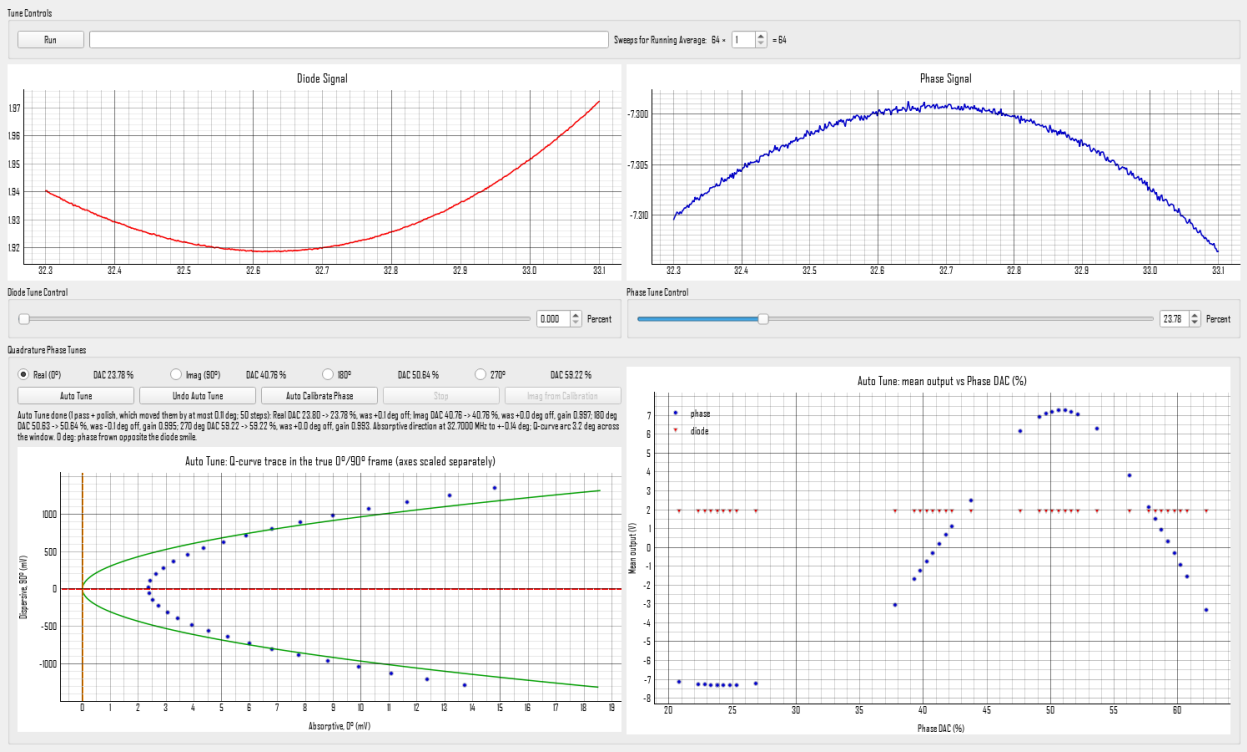


Figure 4: The Tune tab after a cycle4 **Auto Tune** in the simulator. Bottom left: the tune selector with each tune's DAC setting, the **Auto Tune**, **Undo Auto Tune**, **Auto Calibrate Phase**, **Stop** and **Imag from Calibration** buttons, and the result line; under it, once Auto Tune has a result, the Q-curve trace about its point at the centre frequency, in the true 0°/90° frame Auto Tune found (absorptive across, dispersive up, each axis scaled to its data so the bow shows), with the fit (green) and the 0° (red) and 90° (orange) axes dashed. Bottom right: the mean output against the phase DAC for each step of the scan. The simulator's Q-meter is heavily damped, so its trace is nearly a straight line; the Hall B tank traces a visible circle (about 39° of arc in the 2026-10-27 data).

5.1 Tuning procedure

1. Tune the tank (diode) as usual, with the Tune loop running (**Run**).
2. Select **Real (0°)** and set the Phase Tune by eye so the Q-curve looks absorptive. Roughly is enough: within about 30°.
3. Select **Imag (90°)** and set its tune by eye, roughly a quarter turn away. For cycle4, also set 180° and 270°, or leave them empty: with a current calibration (§5.3) PyNMR fills them opposite Real and Imag when cycle4 is turned on.
4. Click **Auto Tune** (§5.2). Check the result line and the plot.
5. Leave the selector on **Real (0°)** when done; runs always restore the Real tune at the end.

Starting values. If the saved session has no usable set of tunes (one is missing, at DAC 0 %, or on the Real tune), PyNMR starts from `default_tunes` in `quadrature.yaml`. These are the settings of the 2026-10-27 Hall B cycle4 run at 32.511 MHz: Real 51.40 %, Imag 44.40 %, 180° 30.39 %, 270° 60.31 %. That run's calibration put them at 0°, −79.9°, −180.0° and +100.0°, so Imag and 270° were about 10° short of quadrature; **Auto Tune** corrects that. A usable saved set (the operator's latest) always wins.

5.2 Auto Tune

Auto Tune refines every tune the current sequence uses (Real and Imag, plus 180° and 270° for cycle4) in one click.

1. It pauses the Tune loop if it is running.
2. For each tune it steps the phase DAC through five settings (−3, −1.5, 0, +1.5, +3 % of full scale), 64 sweeps each, returning to the Real tune between groups.
3. It fits these steps as a local calibration, which gives the complex Q-curve $W(f)$ and the phase of each setting.
4. It fits the trace of $W(f)$ and finds its normal at the centre frequency: the absorptive direction.
5. It moves each tune to the setting at the true angle. If a target lies outside the five settings around its tune, it repeats, centred on the new values (at most three passes).
6. It then runs one *polish* pass: five tighter settings (−1, −0.5, 0, +0.5, +1 %) around the new tunes, which measures the new tunes themselves and gives the final answer. If the polish is stopped or a target falls outside its settings, the main pass's answer is kept and the result line says so.
7. It leaves the box on the tune the selector shows and restarts the Tune loop.

While it scans, each step's sweep is shown on the Diode Signal and Phase Signal plots.

Which absorptive setting is 0°. Both θ and $\theta + 180°$ are absorptive. Auto Tune calls 0° the one whose phase output curves *opposite* to the diode, the JLab convention: a diode smile goes with a phase frown at the in-tune Real setting, where a positive polarization gives a downward line (§3). The 2026-10-27 hall data follow it: the diode was a smile (+229 mV) and the 0° phase a frown (−101 mV), with 180° a smile. Both curvatures are measured on the outer half of the window, clear of the deuteron line; if either is too small to tell, the absorptive setting nearer the hand Real tune is kept.

- In cycle4, if Real was set by hand on the wrong side, Real and 180° swap tunes.
- With a pair, a local scan can't reach half a turn away: Auto Tune keeps the nearer setting and says in red that Real is on the smile side and should be moved about half a turn by hand.
- `autotune: zero_by_curvature: False` in `quadrature.yaml` turns the rule off.

A Real/Imag pair takes one pass of 13 steps plus the 13-step polish, about 50 s on the box. A step is one DAC setting visited automatically (64 sweeps); for the operator it is still one click. In the simulator, a cycle4 set with hand tunes 12–32° off took two passes and the polish (75 steps) and landed every tune within 0.15°, which is the statistical error of the absorptive direction itself.

Reading the result line. For example:

> Auto Tune done (1 pass + polish, which moved them by at most 0.31 deg; 26 steps): Real DAC 25.80 → 23.78 %, was +7.6 deg off; Imag DAC 39.30 → 40.76 %, was −10.9 deg off, gain 0.997. Absorptive direction at 32.7000 MHz to ±0.14 deg; Q-curve arc 3.2 deg across the window.

- *was … deg off*: how far the hand tune was from the true angle.
- *gain*: the detector gain at that tune relative to Real. The shifter's insertion loss varies with the DAC, so values a few per cent from 1 are normal. Large departures mean the 90° angle leans harder on the smooth-gain assumption.
- *to ±… deg*: the statistical error of the absorptive direction.
- *arc*: how far the Q-curve turns across the window. It is a few degrees for a heavily damped circuit and tens of degrees for the Hall B tank. Across that arc the absorptive phase itself changes (Appendix A), so no single tune is exactly absorptive at every frequency.

Undo Auto Tune puts back the tunes it started from. The record of each run is saved as `<event_dir>/quad_cal/autotune_<time>.json`; it is never taken for a calibration.

When Auto Tune refuses or fails. It says so in red on the result line and changes nothing.

- *set the … tune by hand first*, or *the … tune is at the Real tune*: set each tune roughly to its angle.
- *not converged*: a target was still outside the steps after three passes; set the tunes closer and try again.
- It won't start during a Run; stop the Run first.

With the deuteron line on. Most tuning is done on the bare Q-curve. A deuteron line centred in the window adds an even function along the normal and an odd one along the trace, so to first order it doesn't tilt the normal at the centre. If a strong or off-centre line biases the result, list frequency ranges to leave out in `autotune: mask_khz` (kHz from the centre, for example `[[-60, 60]]`).

5.3 Auto Calibrate Phase

Auto Calibrate Phase measures the shifter over its whole range: 21 DAC settings from 0 to 94 %, up and then down, 64 sweeps each, with returns to the Real tune every 10 steps (about 90 s). It fits the phase and detector gain at every setting relative to Real, and the result line gives each tune's angle and gain, the range, and where the shifter stops responding. Use it:

- at the start of a cooldown, or after any change to the RF hardware;
- after the centre frequency moves by more than 0.05 MHz (the calibration is then marked stale);
- to fill 180°/270° for cycle4, or to set Imag with **Imag from Calibration**.

Calibrations are saved in `<event_dir>/quad_cal/<time>.json` during the scan, so a stop or a crash loses nothing; **Stop** ends it after the current step without fitting. Don't close PyNMR until the line says done.

6 Taking quadrature data

6.1 Starting a quadrature run

1. On the Run tab, choose the sequence next to the **Quadrature** checkbox:
   - **pair** 0/90: the default;
   - **abba** 0/90/90/0: cancels a linear drift between the two quadratures;
   - **cycle4** 0/90/180/270: also cancels the detector offsets, Eq. (1).
2. Tick **Quadrature**. If PyNMR refuses (a tune missing, or two tunes at the same DAC setting), the reason appears in red next to the checkbox.
3. Set Sweeps per Event ($64 \times N$) and press **Run**. Events now alternate through the sequence.
4. Stop by hand with **Stop**. PyNMR puts the Config and the box back on the Real tune.

The checkbox starts unticked at every launch. Each event is still one record in the event file, tagged with its angle. The DAC is written for the next event at the end of the previous one, while the box is idle (a write on the run connection just before the sweeps made 640-sweep events time out).

6.2 Baselines

Each angle needs its own baseline: the Q-curve looks different at each reference phase, and so do the detector offsets. Selecting them is as quick as in normal mode, because one selection of a baseline run is split by angle automatically.

Taking a baseline.

1. Finish tuning first (**Auto Tune**). A baseline belongs to the tunes it was taken at.
2. Move the field so the line is out of the window, as for any baseline (same centre and half-width).
3. On the Run tab, choose the sequence you will take data with, tick **Quadrature**, and set the Event Label to Baseline if you like.
4. **Run** for a few complete cycles (for example 4 cycles of 64×10 sweeps), then **Stop**.

Selecting it.

1. On the Baseline tab, click **Select Current Eventfile** (or **Show Recent Baselines** in a later session). The table now has **Angle** and **Phase DAC (%)** columns, right after Time (Fig. 5).

2. Select the whole baseline run in one go: click its first row and shift-click its last. Mixed angles are expected; don't pick them out by hand.
3. Read the line above **Set Baseline**. It says what will be set for each angle (rows, sweeps, DAC setting), and in red what needs attention: an angle with no rows, rows at another tune, or a baseline taken at a setting the tune has since moved from.
4. Click **Set Baseline**. The 0° average becomes the main baseline (as in normal mode) and each other angle gets its own. Each angle's baseline is written to `recent_baselines.txt`, so in a later session **Show Recent Baselines** lists one row per angle: select those rows together and **Set Baseline** again.
5. Check the **Baselines** line on the Run tab, under the Quadrature status: every angle should say ok.

The Baseline tab's plots. While Quadrature is ticked, or a selection holds quadrature rows, the two plots become a 2×2 grid (Fig. 5): the left column is absorptive (0°, with $-1 \times 180°$ on top), the right column dispersive (90°, with $-1 \times 270°$). The top row is Selected Baseline. The bottom row is Current Sweep minus Baseline, using the latest sweep at each angle, so with the line off-window it should be flat noise at every angle, and with the line on it shows each angle's signal. On the top row the $-1\times$ curves sit apart by twice the detector's DC offset, as on the Run tab.

What PyNMR does with the selection.

- Rows are averaged per angle (weighted by sweeps), never across angles.
- Within an angle, only rows at the same phase DAC setting as the latest selected row are averaged, so a selection spanning a retune never mixes tunes; the rows left out are listed.
- Each event subtracts its own angle's baseline. The record says which (`quad_base_ref_deg`), at what setting (`quad_base_vout`), and whether that matches the event's tune (`quad_base_match`).
- An angle with no baseline uses the 0° one, which is wrong for a dispersive record; the Run-tab line shows it in red.
- Selecting an ordinary, untagged baseline (for example one taken with Quadrature off) clears the per-angle ones, and every angle uses it.
- After **Auto Tune** or any change to a tune, that angle's baseline is from another setting: the Run-tab line says "taken at another tune". Take a new baseline set.

6.3 Background subtraction for 90° and 270° records

The Analysis tab's subtraction fits a polynomial to the wings and subtracts it. That suits the absorption, which is small in the wings. The dispersion falls off only as $1/(f - f_L)$, so its tails reach the edges of the window, and a wing fit takes part of them for background. In the 2026-10-27 hall data a cubic wing fit ate the dispersion's tails, while a cubic fitted jointly with the Kramers–Kronig transform of the absorption, over the whole sweep, left a dispersion that matched it to the noise.

PyNMR now does the second for every 90° and 270° record:

$$y(f) = \sum_{j=0}^{3} c_j x^j + s\,\mathcal{H}[A](f), \tag{2}$$

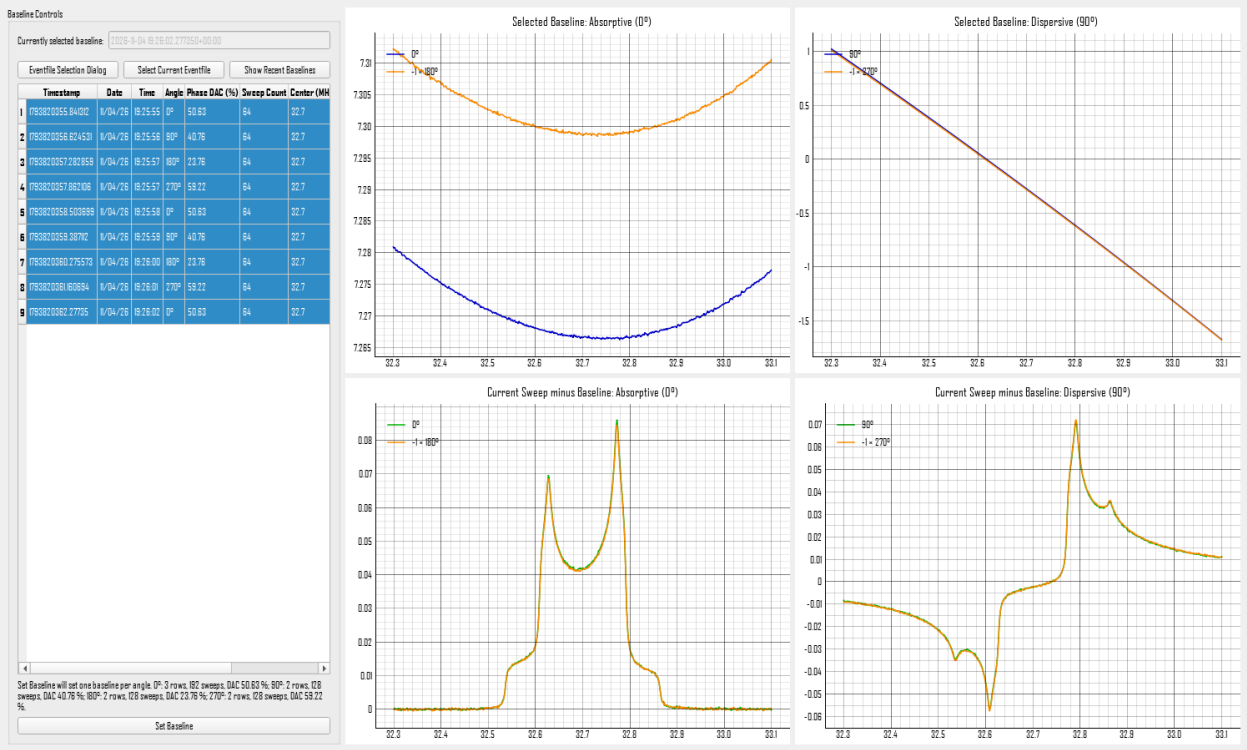


Figure 5: The Baseline tab in the simulator, with a whole cycle4 baseline run (9 rows) selected and set, after the line has come back on. The Angle and Phase DAC columns follow Time. Above **Set Baseline**, the summary lists what each angle gets. Right: the 2×2 grid, absorptive on the left and dispersive on the right; Selected Baseline on top (orange: $-1 \times 180°$ and $-1 \times 270°$), Current Sweep minus Baseline below, where the opposite settings overlay their partners.

where y is the baseline-subtracted sweep, x the frequency scaled to ± 1 across the window, A the absorption of the latest 0° record of the same run (averaged with $-1\times$ the 180° record of the same cycle in cycle4), $\mathcal{H}$ the Hilbert transform, and s a free scale (the gain ratio and the sign of 90° go into it). Only the cubic is subtracted, so Fit Subtracted keeps the whole measured dispersion. The fitted $s\,\mathcal{H}[A]$ is drawn dashed on the Fit Subtracted Imag plot as a check: with a good Imag tune the two lie on top of each other, and $|s|$ is the gain ratio (about 1).

- 0° and 180° records keep the Analysis tab's subtraction.
- The first dispersive record of a run, before any 0° record, uses the wing fit; `quad_sub_method` records which was used.
- The part of the dispersion due to absorption outside the window can't be predicted from inside it; it is smooth and goes into the cubic.
- `dispersion: joint_kk: False` in `quadrature.yaml` goes back to the wing fit for every angle.

6.4 The plots

While Quadrature is ticked, a 2×3 grid replaces the three plots under the Run tab (Fig. 6).

- The columns are the processing stages: Raw Signal, Baseline Subtracted, Fit Subtracted.

- The top row is Real (0°), the bottom row Imag (90°).
- In cycle4, $-1\times$ the 180° record is drawn in orange over every 0° plot, and $-1\times$ the 270° record over every 90° plot, so the opposite settings can be compared at a glance. On the Raw row they sit apart by twice the detector's DC offset; that gap *is* the offset.
- Baseline Subtracted shows the fitted background (yellow), with the points it was fitted to dashed red (§4.3).
- Fit Subtracted: the solid purple line is the Analysis tab's result curve (JLab's; flat at zero for the default *Integrate Full Range*, a fitted lineshape for the fitting methods). The dashed purple on Imag is the fitted KK curve (§6.3).
- A raw curve follows its event live as the sweeps come in. Every other curve changes only when a record at its own angle finishes, so all of them can be watched as the line changes.
- The big polarization number keeps showing the last Real record. The Signal Area box shows one line per angle, each from that angle's latest record, and changes only when a record at that angle finishes.

Unticking Quadrature shows JLab's three plots again.

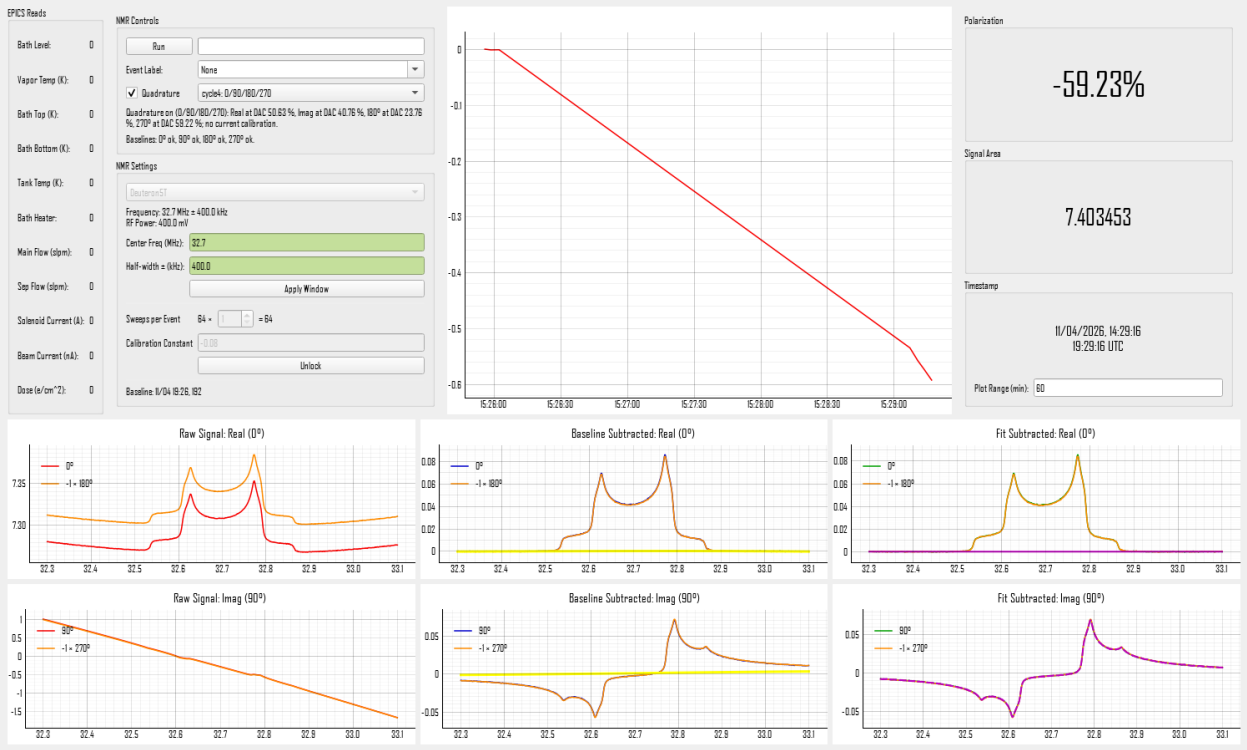


Figure 6: The Run tab during a cycle4 run in the simulator (Sweeps per Event 64×1), after a cycle4 baseline set was selected in one go. Bottom: the 2×3 quadrature grid, with $-1 \times 180°$ and $-1 \times 270°$ in orange. On the Raw row the gap between 0° and $-1 \times 180°$ is the simulated detector offset; each angle's own baseline removes it. Fit Subtracted Imag shows the whole dispersion, tails included, with the fitted KK curve dashed on top of it (here $s = 1.01$).

6.5 Where the records go

- **Event file:** every record, at every angle, with the `quad_*` fields of Table 2.
- **EPICS and history (pol plot, TE tab):** 0° records only. The area and polarization of 90°, 180° and 270° records are not polarizations.
- **Online fitter:** 0° records only.

Field	Meaning
`quad_ref_deg`	nominal angle: 0, 90, 180 or 270
`quad_vout`	phase DAC value for this record (equal to `phase_vout`)
`quad_tunes`	all tunes in use, e.g. {`"0": 0.514, "90": 0.444, ...`}
`quad_mode`, `quad_sequence`	`pair`/`abba`/`cycle4`/`custom`, and the list of angles
`quad_group`, `quad_index`	one pass through the sequence, and the position in it; pair records by group
`quad_state`	`ok`, or `suspended` if the box refused a DAC write (the rest of the run is Real only)
`quad_dac_ok`, `quad_dac_set`	whether the box accepted the write, and which path set the tune
`quad_settle_ms`	wait after the DAC write
`quad_cal_id`, `quad_cal_valid`	calibration in force, and whether it covers these tunes
`quad_cal_angle_deg`, `quad_cal_gain`	fitted angle and gain of this tune relative to Real
`quad_base_ref_deg`	angle of the baseline subtracted (null for an untagged baseline)
`quad_base_vout`, `quad_base_match`	phase DAC the baseline was taken at, and whether it is this tune
`quad_sub_method`	90°/270° records: `joint KK`, or the wing fit and why
`quad_kk_scale`, `quad_kk_curve`	fitted scale s and $s\,\mathcal{H}[A]$ (dispersive records)

Table 2: Quadrature fields on each event record.

Offline. With cycle4, Eq. (1) gives I and Q free of offsets. With a pair at a known angle δ and gain ratio g (from the calibration), $I = P_0 - o_0$ and $Q = \left[(P_\delta - o_\delta)/g - I\cos\delta\right]/\sin\delta$; the noise penalty is only $1/\sin\delta$, so δ need not be exactly 90°. The first hall data (2026-10-27) showed the measured dispersion matching the Kramers–Kronig transform of the absorption to about the noise, once the Q-curve background was fitted jointly with it.

7 Taking and using a TE

A thermal-equilibrium (TE) measurement calibrates the polarization: at a known temperature T and field B the polarization is the Boltzmann value $P_{\mathrm{TE}}(B,T)$, so the calibration constant is $\mathrm{CC} = \langle P_{\mathrm{TE}}/\mathrm{area}\rangle$, and every later event gets $P = \mathrm{area} \times \mathrm{CC}$. The numbers below match the markers in Fig. 7.

1. **Take the TE data** (Run tab). Microwaves off, target at thermal equilibrium at a steady, known temperature; the EPICS temperature channel (`epics_temp`, `TGT:PT12:Bath_Top_T` by default) must be live, because the TE uses its reading for each event. Use the same window, sweeps, baseline and Analysis-tab settings you will use for polarized data: the CC converts *this* area to polarization. Set the Event Label to **TE** (the TE tab lists only events labelled TE or None) and run long enough to average the small signal. In quadrature only the 0° records reach the history, so the TE uses Real records only.

2. **TE tab, Area vs. Time:** set **History to Show (min)** to cover the TE, and drag the blue region over it. The line above reports an exponential fit (relaxation time and asymptote) to judge whether equilibrium was reached.
3. **Select for TE:** the lower plot shows the blue selection; drag the green region over the flat, equilibrium part. The fitted slope is shown above the table and should be consistent with zero.
4. **The points** appear in the table with their area and temperature. Double-click a row to drop an outlier. Points without a temperature reading are refused.
5. **Species and field:** choose **Deuteron** (the box starts on Proton) and enter the field in T.
6. **Calculate TE from Points**: TE Results shows the average TE polarization and CC, with spreads.
7. **Save Results & Use CC**: the report is written to `te_dir` (default `te/`, created if needed) and the CC is put in use. The Run tab's Calibration Constant box shows it, and it is saved in the session.

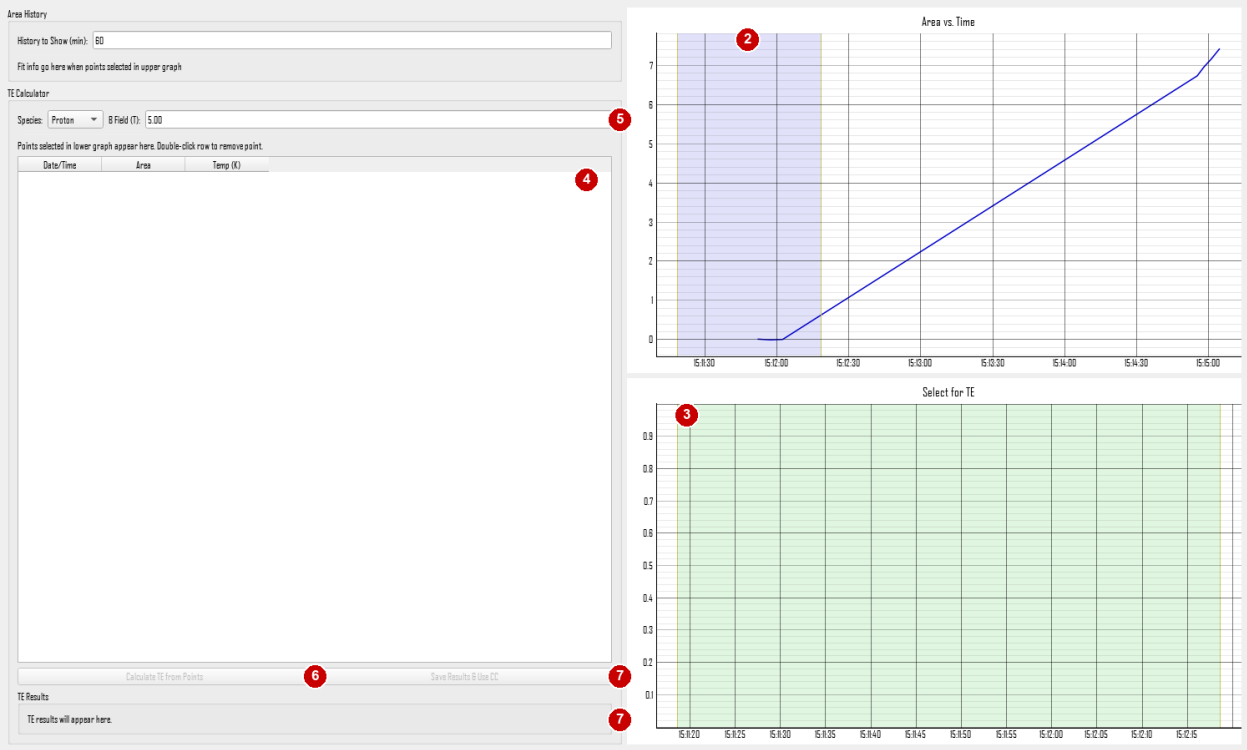


Figure 7: The TE tab (simulator, so no temperatures and no points selected). 2 Area vs. Time with the blue selection. 3 Select for TE with the green selection. 4 the selected points. 5 Species and B Field. 6 **Calculate TE from Points**. 7 **Save Results & Use CC** and the TE Results.

8 The Analysis tab

The Analysis tab sets how every event is analysed, in three stages: subtract a baseline, subtract a fitted background, and turn what is left into a result (the area). Changing anything re-analyses the last event on this tab at once, so the effect can be judged, and applies to every later event; records already written keep what they had. Numbers match Fig. 8.

1. **Baseline Options:** *Baseline Selected from Baseline Tab* (normal), *Polynomial Fit to Wings* (fit the raw sweep's wings instead of using a baseline), or *No Baseline Subtraction*.
2. **Subtraction Options:** *Polynomial Fit to Wings* (normal) or *No Fit Subtraction*.
3. **Polynomial order** (2nd to 8th; 3rd by default) and **Fit bounds (0 to 1)**: the two wing regions as fractions of the sweep. The Run tab's **Wing bins** box sets these bounds (§4.3).
4. **Result Options:** *Integrate Full Range* (default: the area is the sum of Fit Subtracted), *Integrate within Range*, *Peak Height*, *Fit Gaussian and Integrate*, *Fit 2 Gaussians and Integrate*, *Deuteron Peak Fit* (JLab's lineshape fit; slower, up to about 25 s per event), or the two UNH options below.

   Optimized McClellan Fits (Equilibrium) and **(ssRF)** run the online fitter's own fits (the `FitEngine` of LabView-NMR-Fitter, imported from the sibling folder) on each event, fed exactly as the data streamer feeds the fitter.

   - *Equilibrium* is the fast fit: r and the phase float, a cubic is profiled out, and every lineshape constant, *including the amplitude scale*, is held fixed; $P = (r^2-1)/(r^2+r+1)$ (ratio method, no CC needed).
   - **Press Update Constants** once about ten equilibrium lines are in, on a strong, steady line: it runs the full open fit on their average and refreshes all constants. Until then the status line is red and P is not trustworthy (in the simulator the preset amplitude was 1.6 times too small and P came out 1.7 times too high). Constants taken on weak lines (a few %) are poor: re-run it once the signal is strong. The constants are saved in `<event_dir>/mcclellan/` and reloaded at the next start for the same material.
   - *ssRF* is the fitter's hole-burnt fit against the Update-Constants reference; run Update Constants on the equilibrium line the manipulation starts from. Not yet tested inside PyNMR (the simulator burns no holes).
   - Material: ND3, d-butanol or d-propanediol (`unh/mcclellan.yaml`). In quadrature only 0° records are fitted. The fitter's fast fit misreads small *negative* P (around −5 to −15 %) as positive; this is a property of the fitter, reported to its maintainers.
5. **Plots:** Baseline Subtraction (raw, baseline, difference); Fit Subtraction, with the wing regions shaded; Results, with the result curve (the purple line on the Run tab).

- The CC from a TE is valid only for the analysis settings used during that TE. Change them and take a new TE.
- In quadrature, 90° and 270° records use the joint KK subtraction (§6.3) instead of step 2's choice. Re-analysis on this tab uses the tab's own choice for every record, for display.

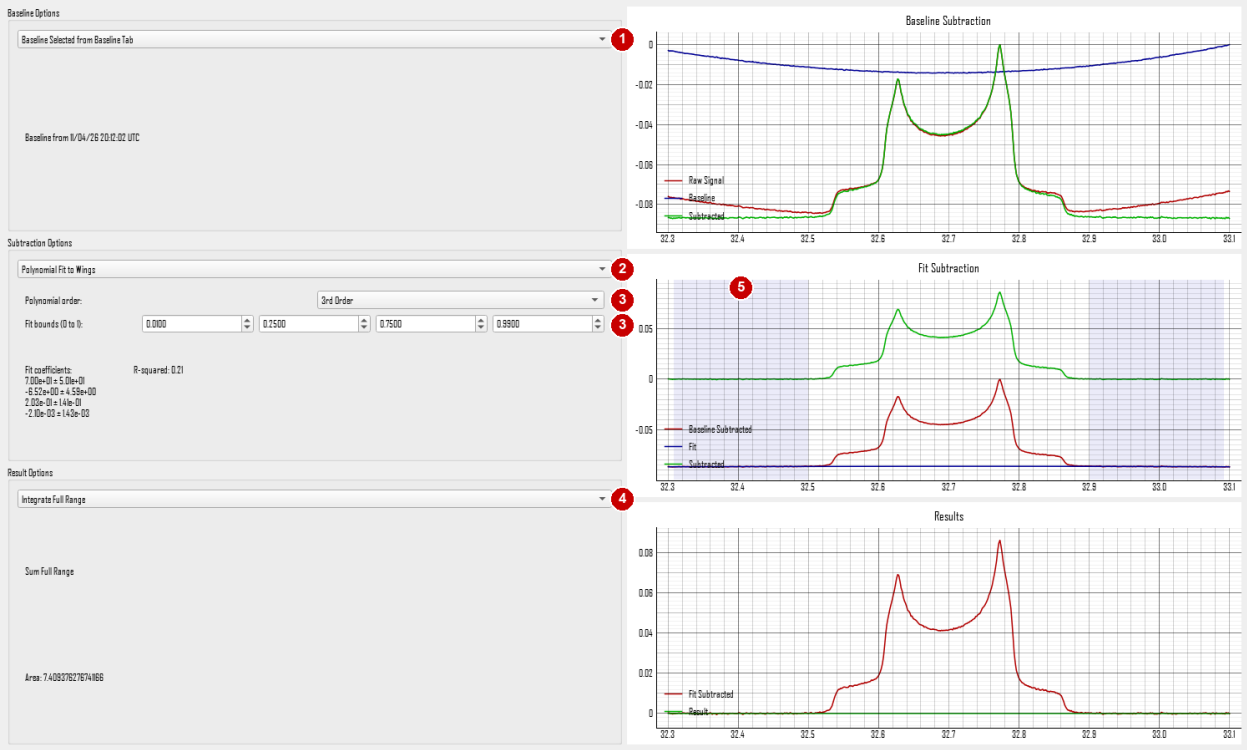


Figure 8: The Analysis tab. 1 baseline method. 2 subtraction method. 3 polynomial order and fit bounds. 4 result method. 5 the shaded wing regions on the Fit Subtraction plot.

9 Troubleshooting

10 Status and limits

- **Tested on the box (2026-10-27):** quadrature alternation, cycle4, the calibration, the per-angle records. 16 complete cycle4 groups were taken on the deuteron line.
- **Simulator only so far: Auto Tune**, the 2×3 plots, per-angle baseline selection, the joint-KK background for dispersive records, the $64 \times N$ sweep controls, 64-sweep tune chunks, the default tunes, and the fix for 640-sweep timeouts.
- The 90° angle rests on the smooth-gain assumption; 0° and 180° don't.
- The absorptive phase turns across the window by the Q-curve's arc. **Auto Tune** makes the reference absorptive at the centre frequency; the offline complex fit handles the rest.

A Why the Q-curve fixes the absorptive phase

For the constant-current (Liverpool) Q-meter, the detected voltage is a fixed function of the tank impedance, $V = F(Z)$ with $F(Z) = UZ/(R_{cc} + Z)$, and $Z = R + iX(f)$. Sweeping the frequency changes the reactance X; absorption by the sample adds to the coil's resistance R (in proportion to χ''). So

$$\frac{dV}{df} = F'(Z)\, i\, X'(f), \qquad \frac{dV}{d\chi''} \;\propto\; F'(Z)\, \frac{dR}{d\chi''}. \tag{3}$$

Symptom	What to do
Quadrature box won't tick	Read the red line beside it: set the named tune on the Tune tab, or move it off the Real tune's DAC value.
Box stops answering ("Exception starting DAQ connection: timed out")	Power cycle the box, then restart PyNMR. It happens when PyNMR dies mid-sweep; check you're on `pynmr312`.
Quadrature suspended mid-run	The box refused a DAC write; the rest of the run is Real only. Check the log, restart the run.
"no current calibration"	The tunes, channel or window changed since the last calibration. The data are fine; run **Auto Calibrate Phase** if you need recorded angles.
Auto Tune not converged	Set the tunes closer by hand (within about 30°) and try again.
Auto Tune result looks wrong	**Undo Auto Tune**. Check the trace plot: a messy trace, or a strong deuteron line, can bias it; try `mask_khz`.
Event of 640 sweeps times out with Quadrature on	Should no longer happen (DAC now written between events); report it with the log.
Baselines line in red	"none": select a baseline set that includes that angle. "taken at another tune": the tune moved (Auto Tune); take a new baseline set.
Imag Fit Subtracted has odd wings	Check `quad_sub_method` is `joint KK` and the dashed KK curve follows the data; a baseline from another tune or a poor 0° record shows up here.
Fitter not updating	Is it running? PyNMR logs one warning per outage and keeps going. Only 0° records are sent.

Table 3: Common problems.

Both share the factor $F'(Z)$; one is multiplied by an imaginary number and the other by a real one. The NMR absorption at frequency f therefore points along the *normal* to the Q-curve's trace at f, and the dispersion along the trace itself. The same holds with the $\lambda/2$ cable between tank and coil, to the extent that the cable's own frequency dependence is small over the window.

Because F is a Möbius map and X is close to linear in f, the trace is a circle (or, for large R_{cc}, a nearly straight line), and the angle about its centre runs as

$$\theta(f) = \theta_0 + 2\arctan\left(\frac{f - f_0}{\gamma}\right), \tag{4}$$

with f_0 the resonance ($X = 0$) and $\gamma = (R_{cc} + R)/X'$ the width of the trace's parametrization, which is wider than the tank's own half-width R/X'. The normal turns with it, so this is also the absorptive phase at each frequency: the form found experimentally by E. Long years ago. Figure 9 checks this in the simulator's Q-meter with a Q ≈ 26 tank like the Hall B one: the normal and the true absorption direction agree to better than 10^{-6} degrees at every frequency, and both turn by about 19° across a ±200 kHz window.

Why the centre frequency and not the resonance. A ±200 kHz window covers a small part of the trace (about $\pm 0.08\,\gamma$ for the Q ≈ 26 circuit, where $\gamma \approx 2.5$ MHz), so f_0 is poorly determined: in a Q ≈ 26 test, the normal taken at the fitted f_0 was 4° off. The normal at the centre frequency needs no f_0 and is where the line sits. Across three test circuits (a straight trace, the Q ≈ 26 circle, and a circle with a $\lambda/2$ cable) and tank detunings up to 200 kHz, it matched the true absorption direction within 0.5°.

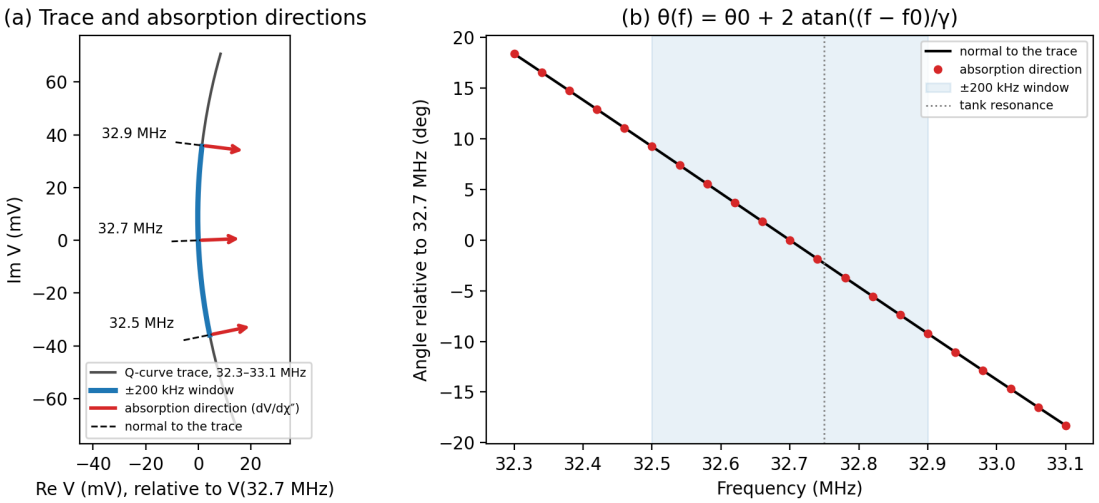


Figure 9: Simulated Q ≈ 26 tank ($R_{cc} = 50\ \Omega$, tank resonance 50 kHz above the window centre). (a) The Q-curve trace with the true absorption direction $dV/d\chi''$ (red arrows) and the normal to the trace (dashes) at three frequencies. (b) Both angles across the sweep, relative to 32.7 MHz.

B The Auto Tune algorithm

Code: `unh/quadrature/autotune.py` (fits) and `AutoTuneThread` in `unh/quadrature/calibration.py`.

1. **Local calibration.** The steps around the tunes are fitted with the calibration model

$$y_{kj} = g(v_k)\,(1 + d\,\tau_k)\big[\mathrm{Re}\,W_j \cos\phi_k + \mathrm{Im}\,W_j \sin\phi_k\big] + o(v_k),$$

   with the phase ϕ free at every DAC value ($\phi = 0$ at the Real tune), g and o low-order polynomials in v, a linear drift d, and W_j the complex response in 32 frequency bins.

2. **Trace fit.** The points $W_j = x_j + iy_j$ are fitted geometrically with a circle written as a conic,

$$A\,(x^2 + y^2) + B\,x + C\,y + D = 0, \qquad B^2 + C^2 - 4AD = 1 \text{ (Pratt)},$$

   which stays finite as the circle straightens into a line ($A \to 0$). The normal at the point for the centre frequency is the gradient, $\theta_{\mathrm{ref}} = \mathrm{atan2}(2Ay + C,\ 2Ax + B)$; its error comes from a jackknife over the bins, folded mod 180°. In the simulator it is within 0.45° of the truth, and the points scatter off the curve by about the noise. (An earlier fit of the circle's parametrization in f followed the trace poorly.)

3. **Targets.** 0° is whichever of θ_{ref} and $\theta_{\mathrm{ref}} + 180°$ is nearer the hand Real tune; 90° is whichever of $\theta_{\mathrm{ref}} \pm 90°$ is nearer the hand Imag tune; 180° and 270° are opposite those.

4. **DAC values.** Around each tune, a quadratic $\phi(v)$ through the five settings is solved for the target. If the answer lies outside them, another pass is centred on it (at most `max_passes`, 3).

5. **Polish.** Once every target lies inside its settings, one more pass with `polish_offsets` (±0.5 and ±1 %) is centred on the new tunes, and its answer is final. The spacing of the settings doesn't set the precision (the fit through them does); the polish measures the final tunes directly and shrinks any error from the shape assumed for $\phi(v)$.

Settings are under `autotune:` in `quadrature.yaml`: `offsets`, `sweeps_per_step` (64), `max_passes`, `polish`, `polish_offsets`, `bins`, `gain_order`, `offset_order`, `fit_drift` and `mask_khz`.